

Convolutional Neural Networks (CNN)

Previous session

In the previous session, you learned:

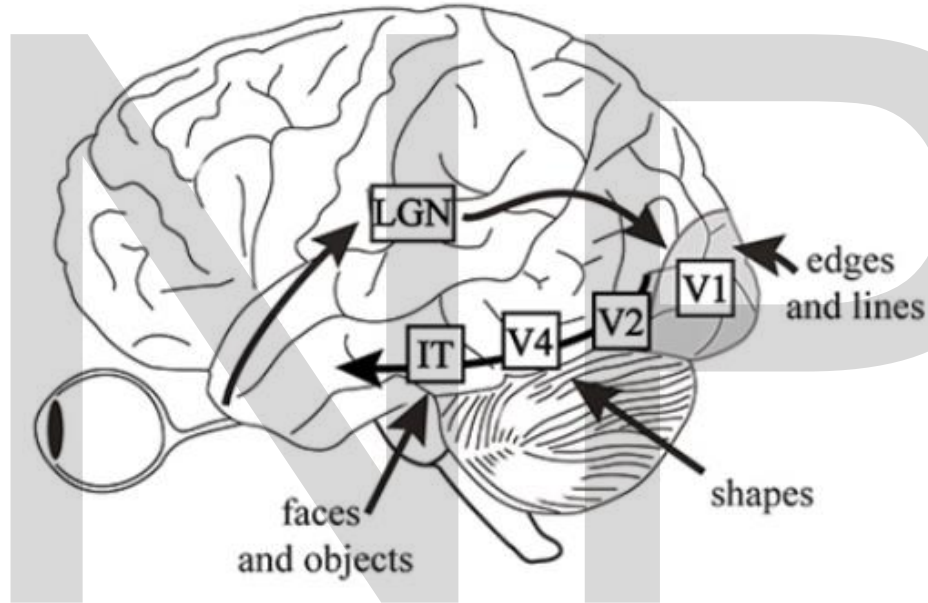
- The **fundamentals of image representation** — grayscale, binary, and ³color images with channels.
- **Why fully connected networks are not suitable for image** data due to the loss of spatial relationships.
- Different **data augmentation techniques** such as rescaling, rotation, flipping, and shifting.
- How to **load and store images for implementation in keras**.

Today's Session Overview

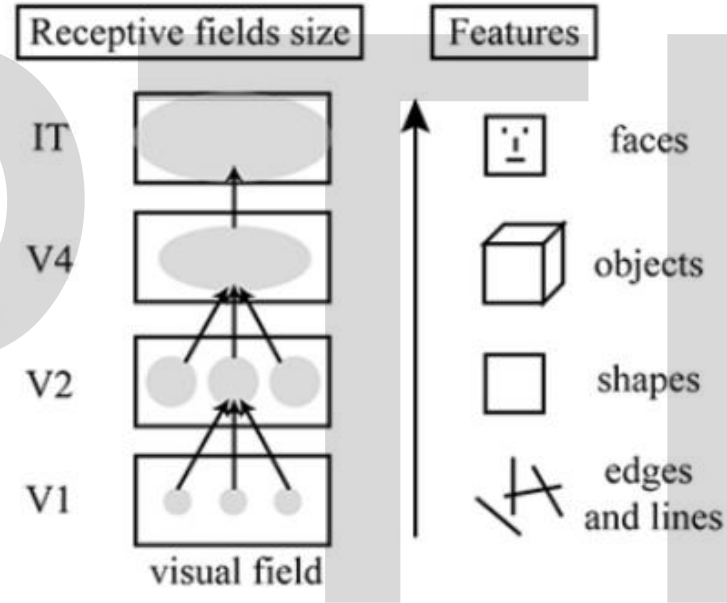
In today's session, we will explore:

- What is the biological inspiration behind the development of Convolutional Neural Networks (CNNs)? – The Human Visual System.
- An **introduction to CNNs**, their purpose, and their role in image understanding.
- The **key components of a CNN**, including convolutional, pooling, and fully connected layers. ✓
- The **convolutional layer in detail**, focusing on filters, feature extraction, and parameter sharing.

Inspiration Behind CNN: Parallels with the Human Visual System



Human Visual Cortex Ventral Pathway



IT (Infero-temporal Cortex):

- A **high-level visual area** responsible for recognizing **faces**, **objects**, and **complex patterns**.
- Represents the final stages of visual recognition.

V2, V4 (Secondary and Tertiary Visual Areas):

- V2 combines low-level features to form **shapes**.
- V4 detects **color** and combines these shapes to form complex **object parts**.

V1 (Primary Visual Cortex):

- Detects **basic visual features** like edges and lines.
- These are the lowest-level features in the visual hierarchy.

Recognizing a Face

1. **V1 (Edges & Lines):** the contour of the eyes, mouth, and face outline.
2. **V2 (Shapes):** circles (eyes), curves (mouth), and lines (nose bridge).
3. **V4 (Objects):** an arrangement resembling a face.
4. **IT (Faces & Objects):** Recognizes the overall pattern as a **specific face**, e.g., your friend's face.

The hierarchical structure of CNNs parallels the human visual cortex

	Brain Area	Function	CNN Equivalent
<i>low-level</i>	V1	Detects edges and lines	Early convolutional layers
	V2	Detects basic shapes and simple patterns	Intermediate convolutional layers
	V4	Detects object parts, color, and features of moderate complexity	Deeper convolutional layers
<i>high-level features</i>	IT	Recognizes complete objects and faces	<u>Fully Connected Layers</u> (after flattening)

CNNs replicate this biological hierarchy — progressing from edge detection to full object recognition, just like the human visual system.

Introduction to CNN

- A convolutional neural network (CNN or ConvNet) is a specialized **deep learning model designed for processing data such as images.**
- It has been widely used in **computer vision tasks** like ***image classification, object detection, and image segmentation.***
- CNNs mimic the human visual system — they focus on **local regions first (lines, edges, corners)** and gradually understand **the whole image through multiple layers.**

First-layer conv filters: local image templates
(Often learns oriented edges, opposing colors)



AlexNet: 64 filters, each 3x11x11

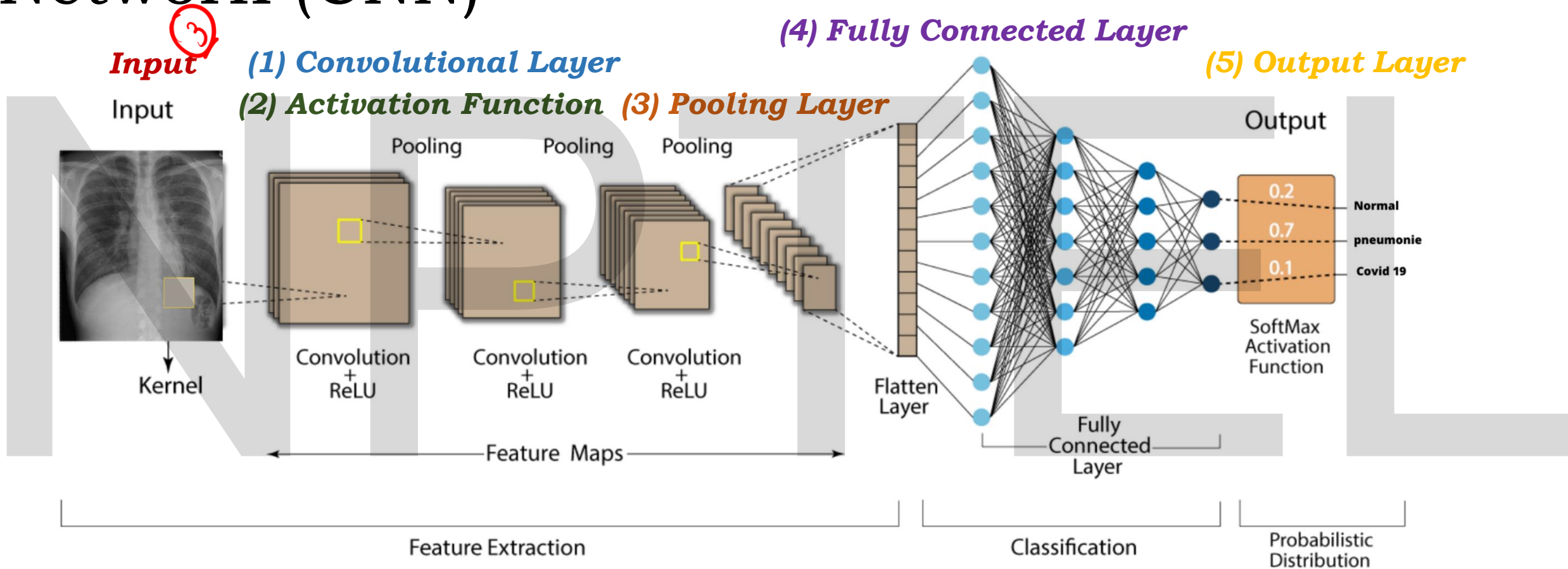
Deeper conv layers: Harder to visualize
Tend to learn larger structures e.g. eyes, letters



6th layer conv layer from an ImageNet model

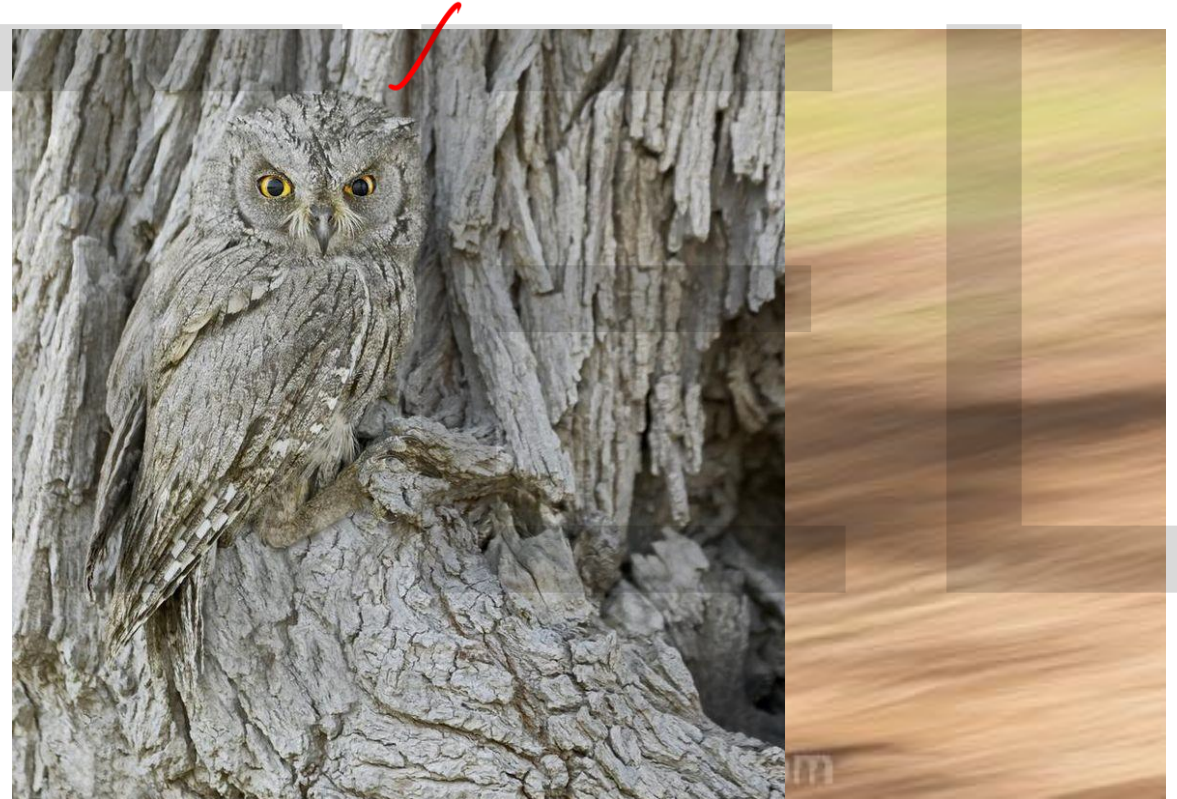
Visualization from [Springenberg et al, ICLR 2015]

Key Components of a Convolutional Neural Network (CNN)



6. Normalization Layer ✓
7. Dropout (Optional) ✓

<https://doi.org/10.3390/electronics11111775>



Convolutional Layer

A **convolutional layer** is the fundamental building block of a CNN that extracts features from input image by applying a set of learnable filters (kernels). It works by sliding these filters over the input data, performing a convolution operation (element-wise multiplication and summation) to create a feature map.

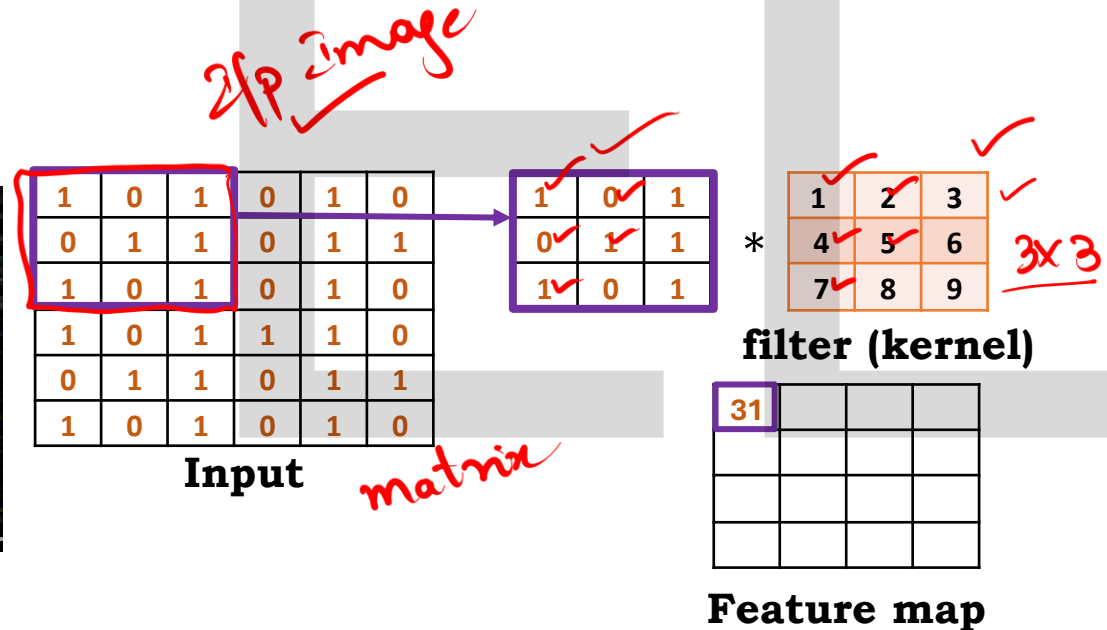
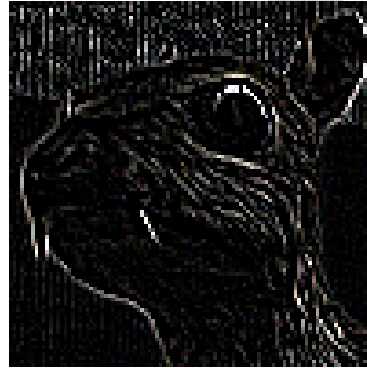
Input image



Convolution Kernel

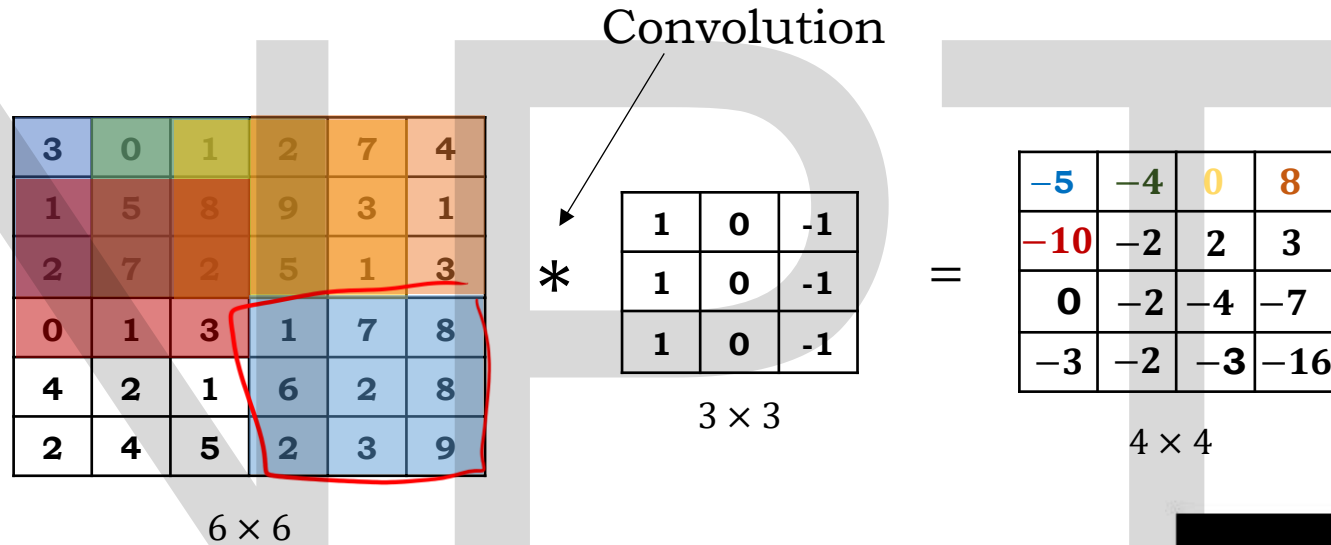
$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Feature map



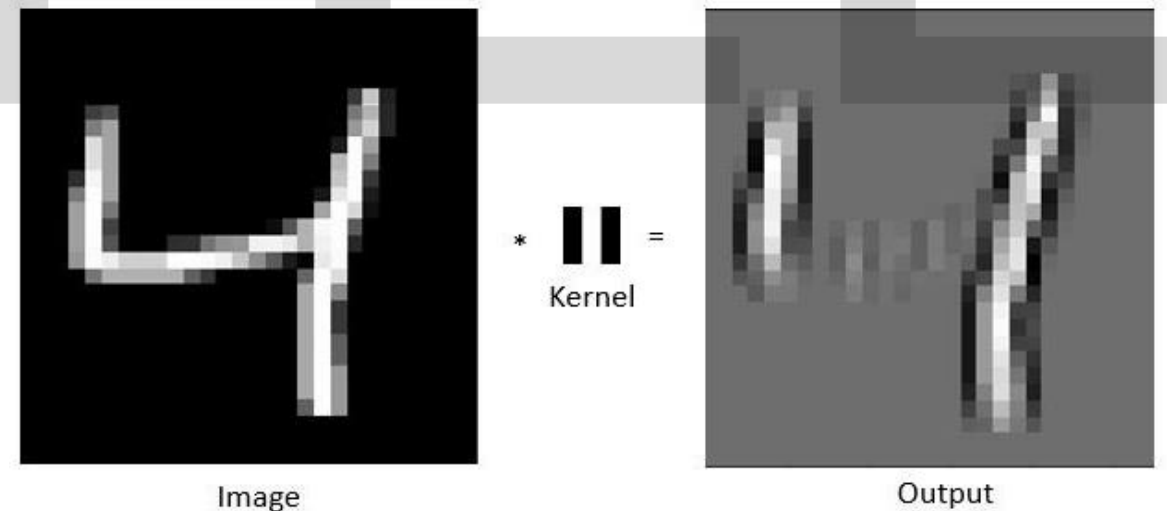
Feature hierarchy: **Early CNN layers** detect **simple features** (edges, corners),
While **deeper layers** capture **complex patterns** (object parts and structures).

CNN: Convolutional operation



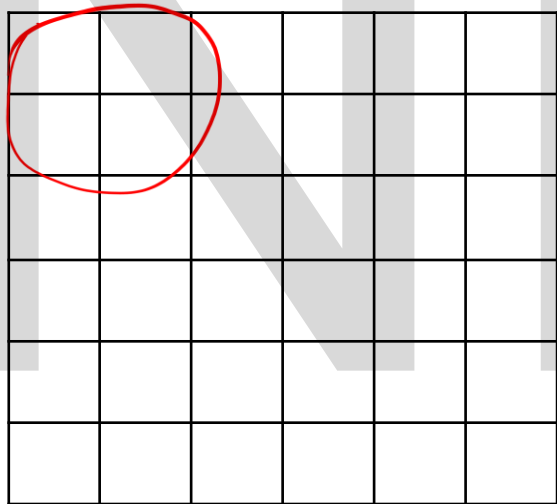
$$(3 \times 1) + (1 \times 1) + (2 \times 1) + (0 \times 0) + (5 \times 0) \\ + (7 \times 0) + (1 \times -1) + (8 \times -1) + (2 \times -1) = -5$$

At each location, the filter performs an **element-wise multiplication** (dot product) and **sums the result**.



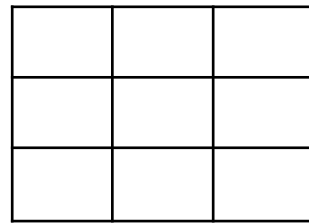
Mathematics of 2D convolution in a CNN

How the **feature map size** is calculated when applying a **convolutional filter (kernel)** to an input image with a stride of 1.



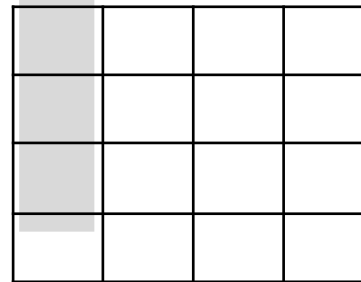
$6 \times 6 (n \times n)$

*



$3 \times 3 (f \times f)$

=



???

4×4

Problem: Feature maps shrink with each layer!

Feature map

Output in general:

$$\begin{aligned} & (n - f + 1) \times (n - f + 1) \\ &= (6 - 3 + 1) \times (6 - 3 + 1) \\ &= 4 \times 4 \end{aligned}$$

Solution: Add padding around the input before sliding the filter

Padding in Convolutional NN

Why Padding is required ?

- During convolution, the size of the output feature map is determined by
- If we simply apply the kernel on the input, then the output feature map will be **smaller than the input**.
- This reduction in size leads to loss of information at the borders of the input image. In order to preserve the border information and maintain the spatial dimension, **we use padding, padding is applied around the input**.

0	0	0	0	0	0	0	0
0							0
0							0
0							0
0							0
0							0
0							0
0	0	0	0	0	0	0	0

*

3×3 (f × f)

=

6×6

$(n + 2p - f + 1) \times (n + 2p - f + 1)$

6×6 (n × n) Input

3×3 (f × f) Kernel

Output feature map
 4×4

Valid and Same convolutions

Valid: No padding

$$(n - f + 1) \times (n - f + 1)$$

Same: Pad so that output size is the same as the input size

$$(n + 2p - f + 1) \times (n + 2p - f + 1)$$

No padding (valid)

```
Conv2D(32, (3,3), padding='valid', input_shape=(64,64,1))
```

Same padding (output height/width same as input)

```
Conv2D(32, (3,3), padding='same', input_shape=(64,64,1))
```

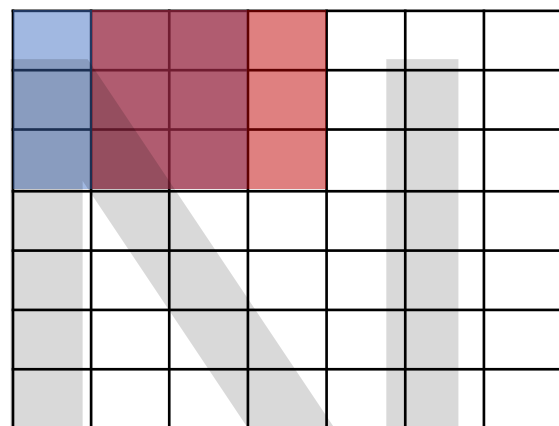
Custom padding with separate layer

```
ZeroPadding2D(padding=(2,2))  
Conv2D(32, (3,3))
```

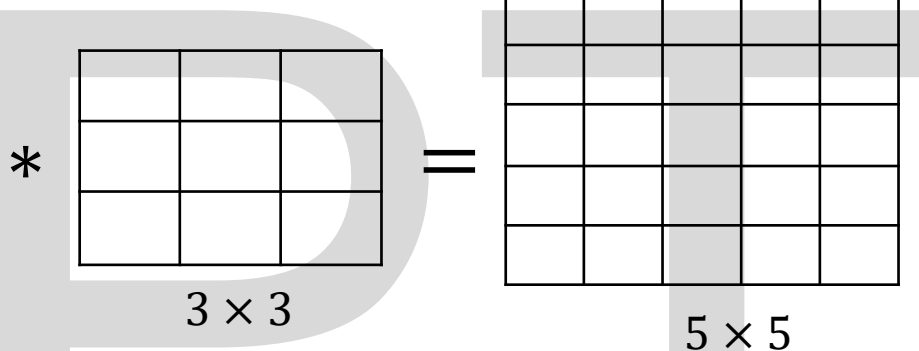
pads 2 rows top/bottom, 2 cols left/right

Stride

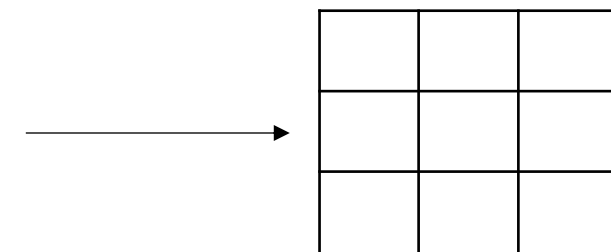
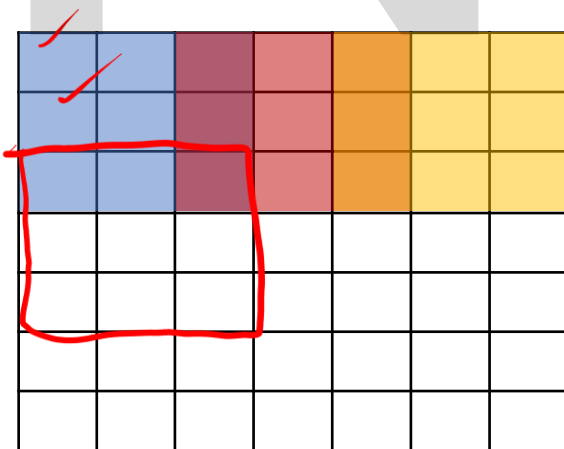
In Keras Conv2D, the strides parameter controls how the filter moves over the input.



7 × 7



$$\frac{7 + 0 - 3}{1} + 1 \times \frac{7 + 0 - 3}{1} + 1 = 5 \times 5$$



Stride = 2

Output = 3 × 3

$n \times n$ image
 $f \times f$ kernel/filter
 p = padding
 s = stride

Then output size is calculated as

$$\frac{n+2p-f}{s} + 1 \times \frac{n+2p-f}{s} + 1$$

$$\left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor$$

$$\lfloor 3.5 \rfloor = 3$$

$$\lfloor 3.1 \rfloor = 3$$

$$\lfloor 3.9 \rfloor = 3$$

The floor function always gives the **greatest integer less than or equal to** the number. ✓

So, both 3.1 and 3.9 “fall down” to **3**

Stride implementation in Keras

```
from tensorflow.keras.layers import Conv2D
```

```
# SAME padding with stride=2
```

```
Conv2D(32, (3,3), strides=(2,2), padding='same', activation='relu')
```

```
# VALID padding with stride=2
```

```
Conv2D(32, (3,3), strides=(2,2), padding='valid', activation='relu')
```

```
# No padding (valid)
```

```
Conv2D(32, (3,3), padding='valid')
```

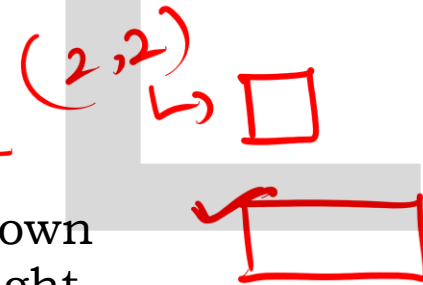
```
# Same padding (output height/width same as input)
```

```
Conv2D(32, (3,3), padding='same')
```

When the input image has different aspect ratio (e.g., 128×256), then the output feature map is rectangular, not square. In such case, stride values differ along height and width, such as

strides = (2, 3)

move 2 pixels down
move 3 pixels right



Stride = 1 which is by default

Summary

- In this session, we explored **the human visual system**, which serves as the **biological inspiration behind the development of Convolutional Neural Networks** (CNNs), where each layer processes visual information hierarchically—from detecting edges to recognizing complete objects.
- We discussed the **key building blocks** of a CNN, focusing on the **Convolutional Layer**, which **extracts meaningful features from images** using **filters (kernels)**.
- The concepts of **padding and stride** were introduced to demonstrate **how convolution operations determine the dimensions of the feature map**.

Overall, this session provided a strong foundation for understanding **feature extraction in CNNs** and **how convolution operations transform input images into feature maps that represent essential visual information**.

Next session

- In the next session, how convolutions are applied to **RGB images** with **multiple channels**. We will examine how **multiple filters** work together to capture diverse features.
- We will study further layers of a CNN — the **Activation (ReLU) Layer**, **Pooling Layer**, and **Flatten Layer**.

NIPTEEL

Thank you